

Laboratório 1 — Infraestrutura como Código com Terraform e Docker

Apostila de apoio — Análise e Desenvolvimento de Sistemas

1. Visão geral do laboratório

1.1 O que você vai fazer

Neste laboratório você vai:

1. Instalar e configurar as duas ferramentas-base de todo o curso: **Docker** e **Terraform**.
2. Escrever seu primeiro arquivo de infraestrutura como código (`main.tf`) e executar o ciclo `init` → `plan` → `apply` do Terraform para subir um container **Nginx**.
3. Evoluir essa infraestrutura para servir uma **aplicação web real** (uma página `index.html` própria, não a página padrão do Nginx).
4. Adicionar um segundo serviço com **estado** — um banco de dados **PostgreSQL** — e entender por que ele é tratado de forma diferente de um serviço como o Nginx.
5. Fechar o ciclo **infraestrutura** → **aplicação** → **dado persistente**, usando apenas `psql` (o cliente de linha de comando do Postgres) para gravar um dado no banco e exibi-lo na página servida pelo Nginx — sem precisar escrever nenhuma linguagem de programação de aplicação.

Cenário: você acaba de entrar como Administrador(a) de Ambientes/Analista DevOps na StackUp Ltda. (empresa fictícia) e sua primeira tarefa é montar o ambiente de testes de uma aplicação nova. Veja o documento "Instruções Gerais" para o cenário completo e as regras de uso de IA como ferramenta de trabalho.

Trabalho em dupla: este laboratório (e os Labs 2 e 3, que continuam o mesmo projeto) é feito em dupla — uma única VM, um único repositório por dupla. Alternem quem digita os comandos e edita os arquivos no VS Code (*driver*) enquanto o outro lê a explicação do "por quê" de cada passo e confere a saída (*navigator*); troquem de papel a cada seção. Isso poupa tempo de configuração (só uma VM precisa baixar Docker/Terraform/VS Code por dupla) e ajuda a não travar em erros de digitação no Terraform. Atenção: as respostas escritas dos exercícios (seção 7) e o material do entregável final (seção 9) devem trazer a contribuição de **cada integrante**, já que a entrega vale nota individual da P1 — não bastam respostas genéricas da dupla.

1.2 Objetivo de aprendizagem

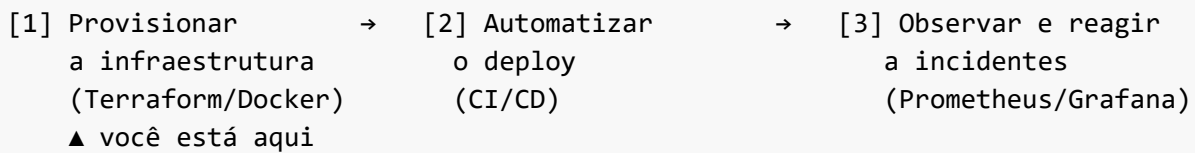
Ao final deste laboratório, você deve ser capaz de explicar e demonstrar:

- Por que se usa uma ferramenta de **Infraestrutura como Código (IaC)** em vez de simplesmente digitar comandos `docker run` manualmente.
- Como o ciclo `init` → `plan` → `apply` protege contra mudanças acidentais em produção.
- A diferença entre um serviço **sem estado** (o Nginx, que pode ser recriado a qualquer momento sem perda de dados) e um serviço **com estado** (o banco de dados, cujo conteúdo precisa sobreviver a recriações do container).

- Como três peças que parecem separadas — servidor web, banco de dados e o arquivo de infraestrutura que os declara — se conectam para formar uma aplicação completa.

1.3 Onde este laboratório se encaixa

Este é o primeiro de três laboratórios da disciplina, que juntos formam a esteira DevOps completa:



No Lab 2, a esteira que você vai construir aqui manualmente (**terraform apply** digitado por você) será automatizada por uma pipeline de CI/CD. No Lab 3, a infraestrutura que você criar aqui ganhará métricas e um dashboard de observabilidade. Ou seja: tudo o que você faz neste Lab 1 é a base sobre a qual os outros dois são construídos — por isso vale a pena entender bem o "porquê" de cada peça, não só o comando.

2. Configuração do ambiente (feita uma única vez por máquina)

Passo 1 — Alocar recursos na VM (com a VM desligada): pelo menos 4 GB de RAM (6 GB ideal) e 2 CPUs.

Passo 2 — Atualizar o sistema

Na linha de comando, digite:

```
sudo apt update && sudo apt upgrade -y
```

Passo 3 — Instalar e configurar o Docker

Por que instalar o Docker? O Terraform, sozinho, não executa nada — ele apenas *declara* o que deveria existir. Quem efetivamente cria, roda e destrói os containers é o **Docker** (o motor de containers). O Terraform, neste laboratório, atua como uma camada declarativa por cima do Docker: sem o Docker instalado, não há nada para o Terraform provisionar.

Na linha de comando, digite — um comando por vez, aguardando cada um terminar antes do próximo:

```
sudo apt install -y docker.io

sudo systemctl enable --now docker

sudo usermod -aG docker $USER

newgrp docker

docker run hello-world # deve exibir "Hello from Docker!"
```

Passo 4 — Instalar o Terraform

Por que instalar o Terraform, se o Docker já roda containers sozinho? Você poderia digitar `docker run nginx` diretamente. Mas isso é **imperativo**: descreve uma ação, uma vez, sem registro do que foi feito. O Terraform é **declarativo**: você descreve o *estado desejado* ("quero um container Nginx na porta 8080") em um arquivo versionável, e a ferramenta calcula sozinha o que precisa criar, alterar ou destruir para chegar lá — sempre da mesma forma, de forma repetível e auditável. Essa diferença é a base de toda a Infraestrutura como Código.

Na linha de comando, digite — um comando por vez:

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install -y terraform

terraform -version
```

Passo 5 — Instalar o VS Code e as extensões de Docker/Terraform

Por que instalar um editor de código, e não só usar nano/vi? Ao longo deste laboratório você vai editar repetidamente três tipos de arquivo: `.tf` (Terraform), `.html` (a página do Nginx) e, no Lab 3, `.yaml` (Prometheus). Um editor de texto puro não avisa quando você esquece uma chave `{ }` ou erra a indentação de um bloco — exatamente os erros que fazem o `terraform validate` falhar. O VS Code, com a extensão certa, sinaliza esses erros **enquanto você digita**, além de oferecer um terminal integrado, para não ficar alternando de janela entre o editor e os comandos `terraform/docker`.

Na linha de comando, digite — um comando por vez:

```
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor >
packages.microsoft.gpg

sudo install -D -o root -g root -m 644 packages.microsoft.gpg
/etc/apt/keyrings/packages.microsoft.gpg

echo "deb [arch=amd64,arm64,armhf signed-
by=/etc/apt/keyrings/packages.microsoft.gpg]
https://packages.microsoft.com/repos/code stable main" | sudo tee
/etc/apt/sources.list.d/vscode.list > /dev/null

rm -f packages.microsoft.gpg

sudo apt update
```

```
sudo apt install -y code
```

Ainda na linha de comando, instale as duas extensões que interessam a este laboratório:

```
code --install-extension hashicorp.terraform  
code --install-extension ms-azuretools.vscode-docker
```

Extensão	Para que serve aqui
HashiCorp Terraform (hashicorp.terraform)	Destaque de sintaxe, autocompletar de blocos resource/provider e formatação automática (terraform fmt) ao salvar o main.tf
Docker (ms-azuretools.vscode-docker)	Painel lateral para ver containers, imagens e volumes rodando — útil para conferir visualmente o que o terraform apply criou, sem depender só do docker ps

A partir de agora, sempre que este roteiro disser "edite o [main.tf](#)" ou "crie o [index.html](#)", abra a pasta do projeto no VS Code ([code ~/meu-projeto](#)) e use o editor e o terminal integrado dele — os comandos de Terraform e Docker continuam exatamente os mesmos.

3. Criando o primeiro projeto (ciclo Terraform)

Na linha de comando, digite:

```
mkdir -p ~/meu-projeto && cd ~/meu-projeto  
code . # abre a pasta no VS Code
```

O VS Code deve abrir com o Explorer (painel lateral esquerdo) mostrando a pasta [meu-projeto](#), ainda vazia.

1. No Explorer, clique no ícone de **New File** (ou [Ctrl+N](#)) e nomeie o arquivo [main.tf](#).
2. Cole o conteúdo abaixo dentro dele.
3. Salve com [Ctrl+S](#) ([Cmd+S](#) no Mac) — com a extensão HashiCorp Terraform instalada, o VS Code já deve colorir as palavras-chave ([resource](#), [provider](#), etc.).

[main.tf](#):

```
terraform {  
  required_providers {  
    docker = {  
      source  = "kreuzwerker/docker"  
      version = "~> 3.0"  
    }  
  }  
}
```

```
}

provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:latest"
}

resource "docker_container" "web" {
  name = "meu-container-web"
  image = docker_image.nginx.image_id
  ports {
    internal = 80
    external = 8080
  }
}
```

Por que Nginx como primeiro serviço? Ele é leve, sobe em segundos e não exige nenhuma configuração para já responder a uma requisição HTTP — o que o torna ideal para você observar, sem distrações, o efeito real de cada comando do Terraform antes de lidar com serviços mais complexos, como o banco de dados da seção 5.

De volta ao terminal integrado do VS Code, digite — um comando por vez, conferindo a saída de cada um antes de seguir para o próximo:

```
terraform init      # baixa os plugins/provedores necessários

terraform plan      # dry-run: mostra o que será criado

terraform apply     # cria de fato (digite "yes")

docker ps

curl http://localhost:8080
```

Bloco	Papel
<code>terraform { required_providers {...} }</code>	Declara quais provedores (plugins) o projeto precisa
<code>provider "docker" {}</code>	Configura a conexão com o Docker local
<code>resource "docker_image" "nginx"</code>	Garante que a imagem exista localmente
<code>resource "docker_container" "web"</code>	Declara o container e seu mapeamento de porta

4. Servindo uma aplicação própria: página customizada via volume

Até aqui, o `curl http://localhost:8080` mostrou a página padrão de boas-vindas do Nginx — útil para confirmar que o container está de pé, mas não é uma aplicação. Para transformar esse container em algo que

sirva **conteúdo seu**, é preciso conectar um diretório do host ao diretório interno onde o Nginx procura os arquivos a servir.

Por que usar um volume, em vez de colocar o `index.html` dentro da imagem? Se o conteúdo fosse "gravado" dentro da imagem do Nginx, qualquer atualização da página exigiria reconstruir a imagem inteira. Com um **volume**, o conteúdo fica em uma pasta comum do host, fora do ciclo de vida do container: você pode editar `site/index.html` e recarregar a página sem precisar recriar nada — o container e a aplicação evoluem de forma independente, o que é essencial quando, no Lab 2, isso passar a ser atualizado automaticamente por uma pipeline.

No VS Code:

1. No Explorer, clique com o botão direito na pasta `meu-projeto` → **New Folder** → nomeie `site`.
2. Clique com o botão direito em `site` → **New File** → nomeie `index.html`.
3. Cole o conteúdo abaixo e salve com `Ctrl+S`:

```
<h1>Minha primeira infraestrutura com Terraform!</h1>
```

4. Abra o `main.tf` (clique nele no Explorer), localize o bloco `resource "docker_container" "web" { ... }` que você já tem e **substitua-o** pelo trecho abaixo, acrescentando o bloco `volumes`. Salve com `Ctrl+S`.

```
resource "docker_container" "web" {
  name = "meu-container-web"
  image = docker_image.nginx.image_id
  ports {
    internal = 80
    external = 8080
  }
  volumes {
    host_path      = "${path.cwd}/site"
    container_path = "/usr/share/nginx/html"
    read_only      = true
  }
}
```

De volta ao terminal integrado do VS Code (menu **Terminal** > **New Terminal**, ou `Ctrl+``), digite:

```
terraform apply
```

```
curl http://localhost:8080 # agora deve exibir seu próprio HTML
```

Use o terminal integrado do VS Code para todos os comandos `terraform`/`docker` daqui em diante — assim você não perde tempo alternando entre janelas.

5. Adicionando um banco de dados: um serviço com estado

Até agora, todo o serviço que você provisionou (o Nginx) é **sem estado** (*stateless*): se você destruir o container e recriá-lo, nada de importante se perde — a página vem do volume, não do container em si. Um banco de dados é diferente.

Por que instalar um banco de dados, e por que ele é tratado com mais cuidado? Um banco de dados guarda **estado**: informação que não pode ser recriada do zero simplesmente reaplicando o Terraform. Se o container do Postgres for destruído sem que seus dados estejam em um volume separado, o conteúdo do banco desaparece junto. Por isso, todo profissional DevOps trata serviços com estado como uma categoria à parte — e é exatamente essa diferença (*stateless* × *stateful*) que este laboratório quer que você sinta na prática, não apenas leia.

No VS Code, abra o `main.tf` e **acrescente ao final do arquivo** (sem apagar o que já existe) o trecho abaixo. Salve com **Ctrl+S** antes de rodar `terraform apply`.

```
resource "docker_volume" "db_data" {
  name = "meu-projeto-db-data"
}

resource "docker_image" "postgres" {
  name = "postgres:16"
}

resource "docker_container" "db" {
  name = "meu-container-db"
  image = docker_image.postgres.image_id
  env   = ["POSTGRES_PASSWORD=senha123"]
  ports {
    internal = 5432
    external = 5432
  }
  volumes {
    volume_name      = docker_volume.db_data.name
    container_path    = "/var/lib/postgresql/data"
  }
}
```

No terminal integrado, digite:

```
terraform apply

docker ps    # confirme que "meu-container-db" está no ar
```

Note o `docker_volume.db_data`: diferente do volume do Nginx (que aponta para uma pasta comum do host), aqui usamos um **volume gerenciado pelo próprio Docker**, o local recomendado para dados de banco — ele garante que, mesmo se você rodar `terraform destroy` e `terraform apply` novamente, o conteúdo do banco sobrevive, desde que o volume não seja destruído junto.

6. Fechando o ciclo: conectando web e banco com `psql`

Nos Labs seguintes você vai automatizar e monitorar essa infraestrutura — mas antes disso, vale fechar o ciclo completo pelo menos uma vez, à mão: mostrar um dado saindo do banco e chegando até a página servida pelo Nginx. Para isso, **não é preciso escrever nenhuma aplicação em Python, Node ou outra linguagem** — o próprio cliente de linha de comando do Postgres, o `psql`, já vem embutido na imagem `postgres:16` que você acabou de subir, e é suficiente para o que precisamos.

6.1 Criando e consultando um dado no banco

Na linha de comando, entre no container do banco e abra uma sessão `psql`:

```
docker exec -it meu-container-db psql -U postgres
```

O prompt muda para `postgres=#` — a partir daqui você não está mais na linha de comando do Linux, e sim dentro do `psql`. Digite os comandos abaixo, um de cada vez, terminando sempre com `;`:

```
CREATE TABLE mensagens (id SERIAL PRIMARY KEY, texto TEXT);

INSERT INTO mensagens (texto) VALUES ('Minha infraestrutura fala com o banco de dados!');

SELECT texto FROM mensagens LIMIT 1;
```

Para sair do `psql` e voltar à linha de comando do Linux:

```
\q
```

Por que esse comando específico? `docker exec -it meu-container-db psql -U postgres` não instala nada de novo — ele apenas abre uma sessão interativa (`-it`) dentro de um container que já está rodando (`exec`), usando o cliente `psql` já presente na imagem oficial do Postgres, autenticado como o usuário `postgres`. É o jeito mais direto de "conversar" com o banco sem sair do ecossistema Docker.

6.2 Levando o dado do banco até a página do Nginx

Agora, sem sair do host (fora do container), rode o mesmo tipo de consulta em modo não-interativo e use o resultado para atualizar o `index.html` que o Nginx está servindo via volume (seção 4):

```
docker exec -t meu-container-db psql -U postgres -t -c "SELECT texto FROM
mensagens LIMIT 1;" | xargs -I{} echo "<h1>{}</h1>" > site/index.html

curl http://localhost:8080
```

O `-t` do `psql` remove cabeçalhos e formatação extra da saída, deixando só o valor da coluna; o resultado é redirecionado diretamente para o mesmo `site/index.html` que o Nginx já está servindo pelo volume — sem precisar de `terraform apply` de novo, porque o volume já está montado. Você acabou de fechar, com três ferramentas (Terraform, Docker e `psql`) e nenhuma linha de código de aplicação, o ciclo **infraestrutura** → **banco de dados** → **página web**.

7. Exercícios de prática — Lab 1

1. **Verificação de ambiente:** execute `docker --version` e `terraform --version` e explique por que essa checagem é boa prática antes de iniciar qualquer projeto.
2. **Leitura do plano:** rode `terraform plan` e explique cada linha marcada com `+` antes de aplicar.
3. **Mudança controlada:** altere a porta externa de `8080` para `9090` e rode `terraform plan` novamente. O Terraform propõe criar ou modificar o recurso? Justifique.
4. **Estado do Terraform:** abra o `terraform.tfstate` e identifique duas informações guardadas sobre o container. Por que esse arquivo não deve ser editado manualmente?
5. **Volume em modo escrita:** mude `read_only` para `false` no volume do Nginx e explique o risco prático de liberar escrita num volume montado em um container.
6. **Drift de configuração:** com a infraestrutura no ar, rode `docker rm -f meu-container-web` (fora do Terraform) e depois `terraform plan`. O que o Terraform detecta e o que ele propõe fazer?
7. **Stateless × stateful na prática:** destrua o container do banco com `docker rm -f meu-container-db` e recrie-o com `terraform apply`. A mensagem que você inseriu na seção 6.1 ainda está lá? Explique o papel do `docker_volume.db_data` nesse resultado — e o que aconteceria se esse volume não existisse.
8. **Atualizando o dado exibido:** insira uma segunda linha na tabela `mensagens` com um texto diferente e repita o comando da seção 6.2 para atualizar a página. O que isso demonstra sobre a diferença entre o *conteúdo* de uma aplicação e a *infraestrutura* que a sustenta?

8. Encerramento do Lab 1

Neste laboratório você provisionou, de forma declarativa e repetível, um serviço sem estado (Nginx) e um serviço com estado (Postgres), entendeu por que cada ferramenta (Docker, Terraform) e cada decisão de projeto (volume comum × volume gerenciado) existe, e fechou o ciclo completo de uma aplicação simples usando apenas `psql`. No Lab 2, essa mesma infraestrutura deixará de ser aplicada manualmente por você e passará a ser aplicada automaticamente por uma pipeline de CI/CD a cada `push` — o próximo passo da esteira DevOps.

9. Para o entregável final

Os três laboratórios desta disciplina terminam em uma **aula de fechamento**, na qual você entrega um único pacote consolidando o que foi feito nos Labs 1, 2 e 3 — essa entrega vale nota da P1. O que este Lab 1 contribui para esse pacote:

- A pasta do projeto tal como está ao final da seção 6: `main.tf` (com Nginx e Postgres) e `site/index.html`.
- As respostas aos 8 exercícios da seção 7.

Guarde bem esse material — no Lab 2 ele será versionado em um repositório Git, e no Lab 3 ganhará mais recursos, até chegar na versão final que você vai entregar.